

Six-Shot Broadcast: A Context-Aware Algorithm for Efficient Message Diffusion in MANETs

Benoît Garbinato, Adrian Holzer, and François Vessaz

Université de Lausanne, Lausanne, Switzerland

{Benoit.Garbinato, Adrian.Holzer, Francois.Vessaz}@unil.ch

<http://www.hec.unil.ch/dop>

Abstract. In this paper, we introduce *six-shot broadcast (6SB)*, a new context-aware message diffusion algorithm that uses location information to fine-tune its broadcasting process. Message diffusion is indeed one of the core challenges brought up by distributed systems and has therefore largely been studied in the context of traditional network structures such as the Internet. With the emergence of mobile ad hoc networks (*MANETs*), new broadcasting algorithm especially geared at these networks have been introduced. These algorithms must reach two conflicting objectives when broadcasting a message, namely reliability vs. efficiency. That is, they must maximize the number of nodes that deliver the message (reliability), while minimizing the number of nodes that forward the message (efficiency). In recent years as more and more mobile devices have become context-aware, several broadcasting algorithms have been introduced using contextual information, such as location, in order to increase reliability and efficiency. Along that line, we provide a in-depth performance evaluation of our 6SB algorithm, by comparing it to similar broadcasting algorithms also targeted at *MANETs*. Our results show that *6SB* competes with the most efficient algorithms in high densities of nodes and offers increased reliability in low densities at a reasonable overhead.

Keywords: MANETs, broadcast algorithm, context-aware systems.

1 Introduction

During the past years we have been witnessing a massive increase of mobile devices. These devices are now ubiquitous and changed the traditional distributed systems from centralized and wired architectures to dynamic, heterogeneous and frequently changing network architectures like mobile ad hoc networks (*MANETs*). These architectures offer many new opportunities for application developers and new challenges for networking protocol designers. The former develop new types of dynamic mobile applications used for example in traffic jam prevention, information dissemination in crowds, strategic data gathering in hostile environments, or peer-to-peer mobile games. The later focus on designing low-level protocols such as broadcast, consensus or atomic commit to create the building blocks for the application developers. In this paper, we introduce *six-shot broadcast (6SB)* as one of these building blocks.

1.1 Mobile Ad Hoc Networks

Mobile ad hoc networks (*MANETs*) are networks of mobile devices without fixed infrastructure. In such a network, every node is directly connected to all nodes located within the range of its radio signal, also referred to as the *technical range*. These nodes are called *neighbors*. Since nodes are mobile, neighborhoods change over time as nodes get in and out of each others technical range.

Communication between neighbors is trivial, as every message emitted by a node is received by all neighbors, thus no routing is necessary. Communication with nodes located outside the neighborhood is a more interesting aspect of *MANETs*, since it implies *multi-hop* message diffusion, where the message must be forwarded by one or more intermediate relays between the sender and the receiver. Henceforth, we will only consider multi-hop message diffusion.

1.2 Message Diffusion in MANETs

In this paper, we address one kind of message diffusion, namely broadcast, where one node sends a message to all nodes in the network. Broadcasting in *MANETs* is made difficult by three of its inherent properties: (1) frequent network configuration change due to the mobility of devices, (2) decentralized architecture due to the absence of fixed infrastructure, and (3) limited resources due to small mobile devices. Such resources include battery and CPU power and the limited network bandwidth, which favors message collisions and might lead to what is called a *broadcast storm* [5] paralyzing the network. In order to evaluate and compare different broadcasting algorithms, two dimensions can be considered:

Reliability indicates the number of nodes in the network that deliver a message compared to the number of nodes that should have delivered the message.

We measure this dimension via the *delivery ratio*. The higher the delivery ratio, the higher the reliability.

Efficiency indicates the cost of broadcasting in terms of message retransmissions. This dimension is measured by the *forward ratio*, the number of nodes relaying a message divided by the total number of nodes in the network. The higher the forwarding ratio, the poorer the efficiency.

The challenge of broadcasting in *MANETs* is to ensure that a high number of nodes deliver the message (reliability), while a small number of nodes retransmit this message (efficiency).

1.3 Contribution and Roadmap

In Section 2, we present a novel context-aware broadcasting algorithm called *six-shot broadcast (6SB)* which constitutes the main contribution of this paper. Section 3 then discusses related work before Section 4 presents the second contribution of the paper, a thorough performance evaluation and comparison of *6SB* and related algorithms. Section 5 wraps up this paper with concluding remarks and hints on future research opportunities.

2 Six-Shot Broadcast

The *six-shot broadcast (6SB)* algorithm is a *context-aware* broadcasting algorithm, which uses location as context information. The *6SB* interface offers the two following typical primitives:

6SB-BROADCAST(m): broadcasts the message m to all nodes located in the network.

6SB-DELIVER(m): acts as a callback when message m is received.

The particularity of *6SB* resides in the scheme it uses to decide whether or not to forward a given message. This scheme is based on a widespread mechanism that we dub *wait and count*. With this mechanism, when a message m is received, a node initiates a waiting time during which it counts retransmissions of m . When the waiting time elapses m is forwarded unless the number of retransmissions is greater than a predetermined *threshold*. *6SB* assigns a different waiting time depending on the geographical location of nodes. The idea is that before a node sends a message m , it associates six geographical *targets* to it as depicted in Figure 1 with Alice as the sender. The number six is chosen because of its interesting property ensuring that every node in the sender's neighborhood can be reached by at least two targets. Among all nodes that receive m , only those located closest to a target should forward m . Note that only nodes located in what is called the *forward zone* can possibly forward messages, all nodes located in the *no forward zone* never forward a message.

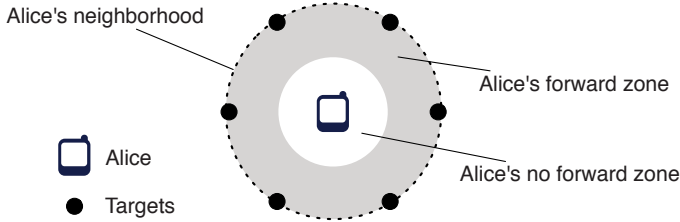


Fig. 1. 6SB principle

2.1 Underlying Services

6SB is built as a primitive that can be used by the application layer and uses two underlying services. As it is location-based it uses an underlying *Location Service (LS)* and like many routing algorithms in *MANETs* it is built upon a *MAC* data link layer.

Location Service. This service allows to access the node's geographical location via a location sensor such as a GPS or location beacons. The location service offers the following two primitives:

LS-GETPOSITION(): retrieves the node's current geographical location.

LS-GETDISTANCE(l_1, l_2): returns the distance between locations l_1 and l_2 .

MAC layer. In *MANETs*, the *MAC* layer is used to broadcast and receive messages in a node's neighborhood. We model this communication capability via the two following primitives:

MAC-BROADCAST(m): broadcasts the message m to all nodes located in the sender's neighborhood.

MAC-DELIVER(m): acts as a callback when message m is received.

2.2 Implementation

At the heart of *6SB* lay the 6SB-BROADCAST and the 6SB-DELIVER primitives presented previously, as well as two important functions, namely GETTARGETS, and GETDELAY. The detail of the *6SB* algorithm is presented in Algorithm 1.

The initialization phase. *6SB* uses three global variables: the *threshold* indicates the number of received retransmissions after which a message does not need to be forwarded anymore. The *nfz* variable indicates the size of the no-forward zone, and the message *counter* list, which is indexed by message IDs and keeps track of the number of received retransmissions (lines 2-5).

The broadcast primitive. When the 6SB-BROADCAST(m) primitive is called with a message m as parameter, a broadcast is initiated. This process adds the location of the sender to the message m before it is broadcasted to the neighborhood via the MAC-BROADCAST primitive. The sender's location is obtained via the location service's LS-GETPOSITION primitive (lines 6-8).

The delivery primitive. When the *MAC* layer receives a message m for the first time through the MAC-DELIVER callback, the 6SB-DELIVER callback is triggered and the counter for m is set to 1. Then the node waits for a delay determined by the GETDELAY function if the delay is valid. During this waiting time, when other copies of m are received, they increment the message *counter*. After the waiting time elapses, m is forwarded using the 6SB-BROADCAST primitive if its *counter* is less or equal to the *threshold* (lines 9-20).

The delay computing function. This function is central to the *6SB* algorithm. When GETDELAY is called, it checks if the node is in the no forward zone. If it is, -1 is returned, otherwise the six targets are retrieved by the GETTARGETS function. Then the DELAY() function computes the node's waiting delay proportionally to its distance to the closest target. We encapsulated this last function since it must be fine tuned according to the application context and the technology used (lines 21-30).

The target computing function. The six targets are computed via the GETTARGETS function by calculating the coordinates of each target based on the location of the sender (the center) and the technical range (lines 31-36).

```

1: uses MAC, Location Service

2: INIT
3:   $threshold \leftarrow \dots$  {threshold fixed by the user}
4:   $nfz \leftarrow \dots$  {size of the no forward zone fixed by the user}
5:   $counter \leftarrow \langle 0, 0, \dots, 0 \rangle$  {list of counters}

6: To execute 6SB-BROADCAST( $m$ ) :
7:   $m.center \leftarrow \text{LS-GETPOSITION}$  {sets the center of  $m$  to the position of the sender}
8:   $\text{MAC-BROADCAST}(m)$  {broadcasts  $m$ }

9: 6SB-DELIVER( $m$ ) occurs as follows:
10: upon  $\text{MAC-DELIVER}(m)$  do {when a message is delivered by the MAC layer}
11:   if  $counter[m] = 0$  then {if  $m$  is received for the first time}
12:     $\text{6SB-DELIVER}(m)$  {delivers  $m$ ,  $\neg$  blocking}
13:     $counter[m] \leftarrow 1$  {initializes the message counter for  $m$ }
14:     $delay \leftarrow \text{GETDELAY}(m)$  {sets the waiting delay}
15:    if  $delay \geq 0$  then {if the delay is valid}
16:      $\text{WAIT}(delay)$  {waits until the delay elapses}
17:     if  $counter[m] \leq threshold$  then {if the threshold is not reached}
18:       $\text{6SB-BROADCAST}(m)$  {forwards  $m$ }
19:   else
20:     $counter[m] \leftarrow counter[m] + 1$  {increments the msg counter}

21: function  $\text{GETDELAY}(m)$  :
22:   $myPosition \leftarrow \text{LS-GETPOSITION}$  {gets the current position of the node}
23:  if  $(\text{LS-GETDISTANCE}(m.center, myPosition) < nfz)$  then
24:   return  $-1$  {returns -1 if the node is in the no forward zone}
25:  else
26:    $distance \leftarrow -1$ 
27:   for all  $t \in \text{GETTARGETS}(m.center)$  do {gets the nearest target}
28:    if  $(\text{LS-GETDISTANCE}(myPosition, t) < distance) \vee (distance = -1)$  then
29:      $distance \leftarrow \text{LS-GETDISTANCE}(myPosition, t)$ 
30:  return  $\text{DELAY}(distance)$  {returns the delay in function of the distance}

31: function  $\text{GETTARGETS}(center)$  :
32:   $targets \leftarrow \emptyset$ 
33:  for  $0 \leq i < 6$  do {creates six targets}
34:    $t_i \leftarrow \langle center.x + \sin(30 + 60 * i) * range, center.y + \cos(30 + 60 * i) * range \rangle$ 

35:    $targets \leftarrow targets \cup t_i$ 
36:  return  $targets$  {returns the targets}

```

Algorithm 1. Six-shot broadcast algorithm

3 Related Work

Many different types of algorithms for broadcasting in *MANETs* exist [2,3,4,5,7,8]. In the following, we investigate the algorithms closest to *6SB*. We first present the *counter-based scheme*, which created the foundation of the *wait and count* mechanism used in *6SB*. Then we investigate four context-aware algorithms, namely the *power-aware message propagation algorithm* and the *optimized flooding protocol*, which fine tune waiting delays according specific locations, similarly to *6SB* and the *location-based scheme* as well as the *area-based beaconless algorithm*, which use location information to fine tune the counting rules.

3.1 Counter-Based Scheme

In the *counter-based scheme (CBS)* [5], when a node receives a new message m , it fixes a random waiting delay before making the forwarding decision. During this delay, the node counts the number of retransmission of m it receives. After the waiting delay has elapsed, the message is only forwarded if the number of retransmissions is less than a predetermined *threshold*. As a result, the forwarding ratio depends on the node density, i.e., in low density areas this ratio will be high, whereas in high density areas it will be very low. Simulation results presented in Section 4 show that this simple algorithm turns out to be very competitive as it does not concede much reliability for highly improved efficiency.

The hop-count aided broadcasting algorithm (*HCAB*) presented in [3] is similar to CBS except that it fine-tunes the counting rule. Messages in *HCAB* store the number of hops they perform in a variable that we call *hopNb*. When a node receives a message for the first time it gets the value v of the message's *hopNb*, sets a random waiting time and then *only* counts retransmissions of the message with *hopNb* greater than v . Then similarly to *CBS* when the waiting time elapses, the message is forwarded if the retransmission counter is below the *threshold*.

3.2 Power-Aware Message Propagation Algorithm

The *power-aware message propagation algorithm (PAMPA)* [4] has the exact same mechanism as *CBS* except that the delay is not fixed randomly but it is based on the intensity of the signal at which a message is received. The stronger the signal, the longer the delay. The idea behind *PAMPA* is that forwarders should be located as far away as possible from the source node. With its mechanisms, *PAMPA* ensures that only neighbors located on the outskirts of a node's neighborhood will forward messages.

3.3 Optimized Flooding Protocol

The *optimized flooding protocol (OFP)* [8] is a context-aware broadcasting algorithm which aims at providing a highly efficient message forwarding scheme.

OFP transposes the broadcasting problem in MANETs into the following geometric optimization problem: *how can we minimize the number of circles it takes to cover a certain surface under the constraint that every circle must have its center on the area covered by another circle?* The answer to this problem is to divide the surface into hexagons the size of a circle, and then to draw a circle on each summit. Transposed back to broadcasting in MANETs, the center of each circle represents the ideal location for forwarders. Figure 2 depicts this idea with a central source node broadcasting a message, which will first be forwarded by three first-hop forwarders and then by six second-hop forwarders and so forth. This solution allows to have the absolute minimum number of forwarders in order to reach all nodes within a surface in an ideal case. The *OFP* algorithm works also like *CBS* except that the waiting time is set according to the distance of each node with the closest ideal forwarding spot. The closer to the spot, the shorter the waiting time.

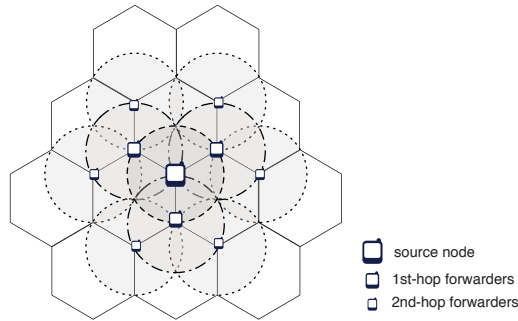


Fig. 2. OFP principle

3.4 Area-Based Algorithms

In the previously presented context-aware algorithms, context is used to fine-tune the waiting time. In other algorithms, such as the *location-based scheme (LBS)* presented in [5] and the *area-based beaconless algorithm*, abbreviated *ABBA* [7], primarily use context to fine-tune the counting rule. In *ABBA* and *LBS* when a message is received for the first time, a waiting time is set either randomly or according to the proximity of the sender very much like previously presented algorithms. The major particularity of *ABBA* and *LBS* resides within their counting rule. Instead of counting the number of retransmissions received, nodes use the location of senders to determine what proportion of their neighborhood is covered by the senders' neighborhoods. If the proportion is less than a threshold value, the message is forwarded.

3.5 Summary

All algorithms presented above use the *wait and count* mechanism introduced by *CBS*. *OFP*, *PAMPA* are the closest efforts to *6SB* in that they also add a

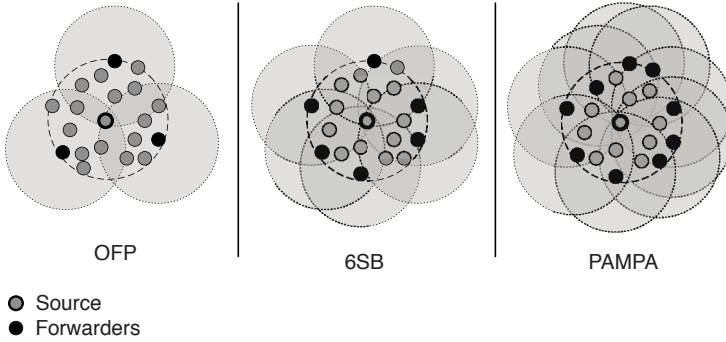


Fig. 3. Forwarding patterns

context-based scheme in order to delegate the forwarding task to nodes located nearby strategic positions. Each of these algorithms defines these positions differently. *PAMPA* selects nodes located closest to the limit of its neighborhood, *OFP* selects nodes closest to three targets, and *6SB* selects node closest to six targets. Figure 3 illustrates each algorithm’s forwarding pattern.

The main difference between these algorithms in terms of forwarding pattern is the number of potential forwarders used. *Pampa* has the highest number and *OFP* the lowest number. In the following section, we compare the performance of these algorithms in order to measure the difference in efficiency and reliability that this difference implies.¹

4 Performance Evaluation

We measure performance in terms of forward and delivery ratios, and compare them to the algorithms presented in the related work (Section 3). Our results change depending on the density of nodes in the field. In high densities, *6SB* performs as well as *OFP*, which is the best algorithm. In low densities, *6SB* performs better than *OFP* in term of reliability for small overhead in terms of efficiency. Our results also demonstrate that minor mobility has no influence on the performance results. Before discussing these results in detail, we present the simulation settings used for the evaluation.

4.1 Simulations Environment

To compute the delivery and forward ratios of the algorithms, we implemented them using the Java-based Sinalgo framework [6] and ran extensive simulations. The context we simulate is one of an event, such as a conference or a festival, where every participant has a mobile device connected to a MANET with IEEE

¹ This difference also implies variations in message diffusion speed, which we do not analyse in this paper.

802.11n WiFi links and a GPS chip. An example of such a device is Apple's iPhone or Nokia's E71. In our context, users are static or moving around at walking speed.

Density and Connection Degree. We define three parameters to characterize the density and the degree of connections of the network: the *map size*, the *technical range* and the *number of nodes*. We choose a square map of 200 meters width with a technical range of 20 meters.² In every simulation, one node broadcasts a message. Each simulation was run 100 times in an initially fully connected network of 160 up to 2000 nodes. This means that every node has an from around 4 up to 60 neighbors. The map shape we use for the simulations is a torus to avoid having to deal with special conditions at the field limits. Table 1 summarizes the general simulations parameters we use.

Table 1. General simulations parameters

Parameters	Value
<i>Map size</i>	200 m · 200 m
<i>Map shape</i>	torus
<i>Map area</i>	40000 m ²
<i>Technical range</i>	20 m
<i>Number of node</i>	160 – 2000
<i>Number of sender</i>	1

Mobility. There exist several mobility models and many studies about their realism in MANETs. One of the most used is the Random Waypoint Model [1]. Our torus-shaped map resolves a known issue of the Random Waypoint Model, i. e., the fact that node tend to converge to the center of the simulation field. In our context, we defined a scenario with mobility at walking speed. So, we set the speed of our Random Waypoint model uniformly distributed between 1 and 2 meters per second. In this mobility model, nodes choose a random waypoint on the field towards which they move. When the waypoint is reached, node wait for a random uniformly distributed random delay from 0 to 10 seconds before choosing another waypoint. The message transmission speed used in the simulation is 0.1 second and represents the time between a MAC-BROADCAST and a MAC-DELIVER in a 20 meter range. Table 2 summarizes the mobility and time parameters used.

4.2 6SB as Good as OFP in High Densities

Figure 4 and 5 present respectively the delivery and forward ratios of the *CBS*, *OFP*, *6SB* and *PAMPA* algorithms in a static setting *without mobility*.

² Note that IEEE 802.11n allows outdoor communications up to 140 meters between laptops and wireless routers and 70 meters for indoor communication. After testing the range of various smart phones, we found values between 20 and 30 meters to be the most reasonable.

Table 2. Mobility parameters

Parameters	Value
<i>Mobility</i>	Random Waypoint in a torus
<i>Delay</i>	Uniform 0 - 10 s
<i>Nodes speed</i>	Uniform 1 - 2 m/s
<i>Message transmission time</i>	0.1 second

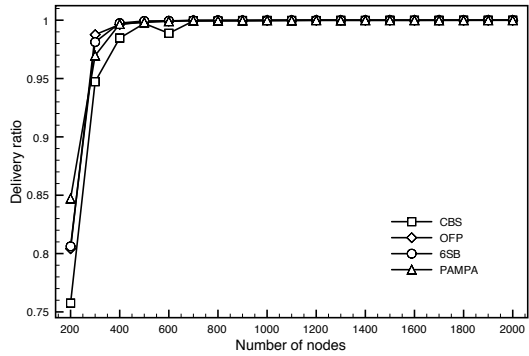


Fig. 4. Delivery ratio in high densities without mobility

These results show that all evaluated algorithms perform well, between 5% to 40% of the nodes have to forward the messages. In high densities (more than 600 nodes), the different algorithms have no influence on the delivery ratio that is equal to 1, because there is enough nodes to propagate the message to the whole networks. The only difference is the forward ratio. The *OFP* algorithm is the one with the best forward ratio, closely followed by *6SB*. These two algorithms are clearly the best in terms of efficiency. *CBS*, despite its simplicity, turns out to be also very competitive. The *PAMPA* algorithm, which is more complicated, demonstrates poorer performances. Figure 6 and 7 present the same results in a mobile context. Mobility has almost no influence on the results. This is due to the significant difference between the slow walking speed at which nodes move and the high message propagation speed.

These results convey the fact that in high densities, when nodes have around 18 or more neighbors (above 600 nodes in our simulation settings), all evaluated algorithms are reliable and *6SB* performs as well as *OFP*, which is the best algorithm in terms of efficiency. Mobility has no influences on the results.

4.3 6SB Better Than OFP in Low Densities

Figure 8 and 9 present respectively the delivery and forward ratios of *CBS*, *OFP*, *6SB* and *PAMPA* in low densities (until 350 nodes). *PAMPA* and *6SB* have the highest delivery ratios. *CBS* has a delivery ratio, which is always the lowest, but it tend to join *PAMPA* and *6SB* in higher densities. *OFP* has an intermediate delivery ratio. It begins at the same low value than *CBS*, but becomes reliable

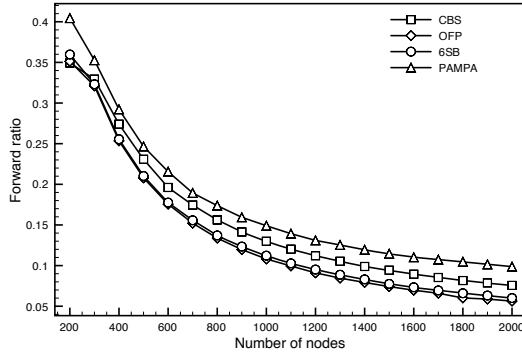


Fig. 5. Forward ratio in high densities without mobility

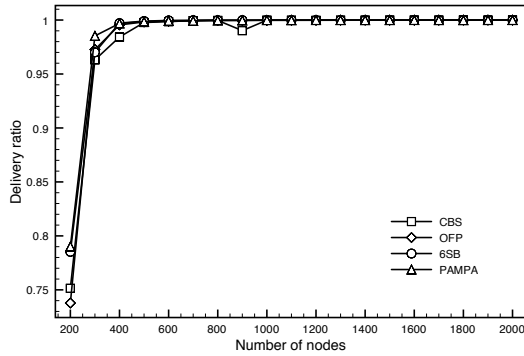


Fig. 6. Delivery ratio in high densities with mobility

more rapidly. In terms of efficiency, *OFP* has always the lowest forward ratio. *CBS* has a forward ratio a little bit higher, followed then by *6SB* and *PAMPA* in the last position. When the density increases, these ratios tend to change; *6SB* became more efficient than *CBS*.

These results show clearly that there is a tradeoff between reliability and efficiency in lower density settings. *PAMPA* and *6SB* focus on reliability, since they have a higher delivery ratio for a higher forward ratio. *OFP* and *CBS* focus on efficiency, with a lower forwarding ratio but also lower a delivery ratio.

In order to measure this tradeoff, we have computed the number of delivers gained with one more forward in comparison to *OFP*, which is the best algorithm in term of forward ratio (see Figure 10). This is the *utility* of having one more forwarder. *CBS* is the worst algorithm because it has the lowest gain for the higher variability of the results. *PAMPA* has a small utility just over 0. *6SB* has the best utility with values above 1. This means that for one more forward, *6SB* will generate more than one extra deliver. So *6SB* allows to have a better delivery ratio than *OFP* for the lowest overhead in term of forward. *6SB* offers the best tradeoff between reliability and efficiency in lower densities.

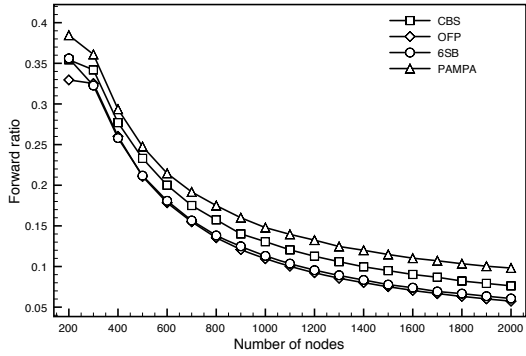


Fig. 7. Forward ratio in high densities with mobility

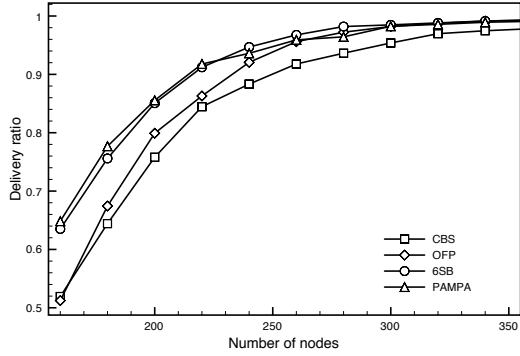


Fig. 8. Delivery ratio in low densities

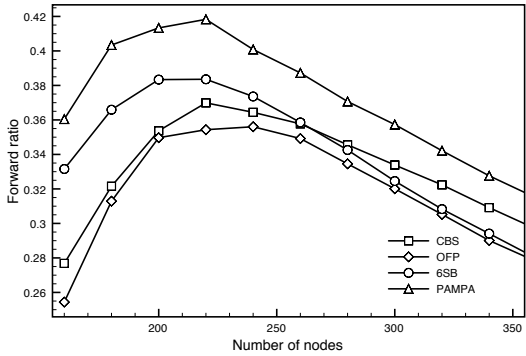


Fig. 9. Forward ratio in low densities

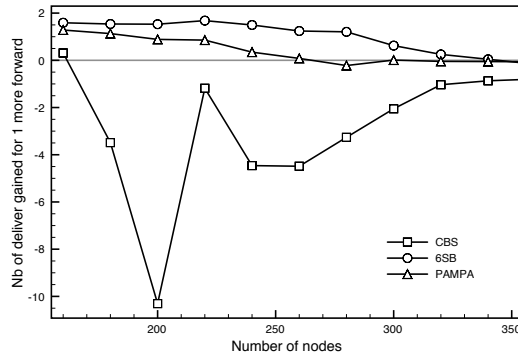


Fig. 10. Number of deliver gained for one more forward in comparison to OFP

5 Concluding Remarks

Finding the right algorithm for broadcasting in a MANET is not a trivial task. The problem of broadcasting in MANETs is a tradeoff between reliability and efficiency. Reliability is measured by the number of nodes that deliver the message and efficiency by the number of nodes that forward the message. A maximum of nodes have to deliver the message, while we try to minimize the number of nodes that forward this message. By optimizing the number of forwarders, we save resources, such as battery power, but we increase the risk that some nodes do not receive the message.

The main contribution of this paper is *six-shot broadcast (6SB)*, a context-aware communication algorithm using location information to fine-tune its forwarding process. Our extensive performance evaluations show that *6SB* competes with the most efficient algorithms in high densities and offers increased reliability at a reasonable price in low densities.

We intend to extend this research in two directions, first we will have a look at a practical implementation of the *6SB* algorithm and its use on mobile devices for a concrete mobile application. Second, we will further examine *6SB* theoretically by conducting simulations in different contexts and by integrating the *6SB* algorithm into other broadcast-based algorithms, such as *FIFO-broadcast*, or *atomic-commit*.

References

1. Camp, T., Boleng, J., Davies, V.: A survey of mobility models for ad hoc network research. *Wireless Communications and Mobile Computing (WCMC): Special issue on Mobile Ad Hoc Networking: Research, Trends and Applications* 2(5), 483–502 (2002)

2. Heinzelman, W.R., Kulik, J., Balakrishnan, H.: Adaptive protocols for information dissemination in wireless sensor networks. In: *MobiCom 1999: Proceedings of the 5th annual ACM/IEEE international conference on Mobile computing and networking*, pp. 174–185. ACM, New York (1999)
3. Huang, Q., Bai, Y., Chen, L.: Efficient lightweight broadcasting protocols for multi-hop ad hoc networks. In: *Personal, Indoor and Mobile Radio Communications, 2006 IEEE 17th International Symposium*, pp. 1–5 (2006)
4. Miranda, H., Leggio, S., Rodrigues, L., Raatikainen, K.: A power-aware broadcasting algorithm. In: *Proceedings of the 17th annual IEEE international symposium on personal, indoor and mobile radio communications (PIMRC 2006)*, pp. 1–5 (2006)
5. Ni, S.-Y., Tseng, Y.-C., Chen, Y.-S., Sheu, J.-P.: The broadcast storm problem in a mobile ad hoc network. In: *MobiCom 1999: Proceedings of the 5th annual ACM/IEEE international conference on Mobile computing and networking*, pp. 151–162. ACM, New York (1999)
6. Distributed Computing Group of ETH Zurich. Sinalgo: a simulation framework for manets, <http://dgc.ethz.ch/projects/sinalgo/>
7. Ovalle-Martinez, F.J., Nayak, A., Stojmenovic, I., Carle, J., Simplot-Ryl, D.: Area-based beaconless reliable broadcasting in sensor networks. *IJSNet* 1(1/2), 20–33 (2006)
8. Paruchuri, V., Duresi, A., Jain, R.: Optimized flooding protocol for ad hoc networks. In: *CoRR*, cs.NI/031103 (2003) (Informal publication)